

Auto clicker detection with neural networks

xEcho1337
Italy

Requiem
Spain

Abstract

In recent years, the use of automated tools, such as auto clickers, has raised significant concerns in the realm of online gaming, particularly within competitive environments. These tools, designed to automate clicks, have evolved a lot trying to emulate human patterns, and can undermine the integrity of gameplay and disrupt the fairness of competitions among players. In response to this issue, the present project aims to develop a detection system for auto clickers using an artificial intelligence-based approach.

INDICES

0. Abstract: A concise summary of the research problem, objectives, methodology, and key findings.
1. Introduction: Why detecting auto clickers in videogames is important, overview of how auto clickers work.
2. Related work: Overview of auto clicker detection techniques already made by other anti-cheat developers.
3. Methodology: What architecture we did use, how we collected the data, how we pre-processed the data, how did we train and validate the neural network, what features did we use.
4. Results: Performance results, model accuracy and loss, feature importance, evaluation metrics.
5. Discussion: Future work, strengths of the neural network approach, current limitations and challenges.
6. Conclusion: Summarize the findings and their implications for auto clicker detection in several games, potential applications of the work.
7. References: Cite all academic papers, the math formulas, tools used in the research, etc...

1. INTRODUCTION

The rise of online multiplayer games has heightened the importance of fair play, yet the prevalence of automation tools like auto clickers poses a significant threat to this integrity. Auto clickers

enable players to automate repetitive tasks and perform rapid inputs, giving them an unfair advantage and disrupting competitive balance. This not only undermines the gaming experience for legitimate players but also leads to broader issues, such as reduced trust in game developers and potential loss of player engagement.

Auto clickers operate by simulating mouse clicks or keystrokes at speeds unattainable by human players, often using customizable settings to optimize their performance. Their advanced functionality allows them to mimic human behavior closely, making detection challenging. Traditional methods of detection based on simple thresholds are often insufficient, leading to a need for more sophisticated approaches.

This research proposes a neural network-based solution for detecting auto clickers by analyzing user input patterns. By employing a data-driven, supervised learning methodology, the study collects click data from both human players and auto clickers, focusing on features like click intervals and burst patterns. The effectiveness of the model is evaluated using standard performance metrics to ensure it can accurately differentiate between legitimate and automated inputs.

2. RELATED WORK

In the field of auto clicker detection, research has primarily been divided between traditional non-AI methods and more recent AI-driven approaches. Non-AI methods typically involve rule-based sys-

tems or heuristics, which are designed to detect anomalous click behavior by analyzing patterns such as speed, regularity, and timing of inputs. These methods provide clear explanations and transparency, making them relatively easier to understand and implement. However, they tend to lack the adaptability needed to cope with sophisticated auto clickers, which can be programmed to mimic human behavior closely

In previous works, convolutional neural networks (CNNs) demonstrated success in properly detecting auto clickers. Overall, both non-AI and AI-based systems contribute to the current landscape of auto clicker detection, with AI offering greater adaptability but at the cost of complexity and explainability.

3. METHODOLOGY

3.1 Strategy

There are two main strategies for detecting auto clickers in video games: reaction-based detection and click randomization detection.

Reaction-Based Detection examines how players respond to in-game stimuli (like visual or auditory cues). Human players show variable response times due to cognitive processes and decision-making, while auto clickers produce consistent, uniform responses, making them easier to identify.

Click Randomization Detection analyzes the timing and distribution of clicks. Human clicking patterns are inconsistent due to factors like fatigue, whereas auto clickers, even if randomized, lack the same organic variability. This method is beneficial for modeling clicking behavior and distinguishing between human and automated inputs.

The research favors click randomization for training detection models because human clicking behavior is more unpredictable, making it difficult to build robust models based solely on reaction times. Click randomization allows for clearer analysis of differences between human and auto clicker inputs and enables simpler feature extraction, like average click intervals.

As auto clickers become more advanced, they may try to imitate human responses, but click randomization techniques can still reveal their predictability. Overall, focusing on click randomization enhances the effectiveness of detection systems, promoting fairness in competitive gaming.

3.2 Architecture

The neural network used in this study is a Feedforward Neural Network (FFNN). This architecture is well-suited for classification tasks like auto clicker detection, where the objective is to classify click behavior as either human or automated. The model consists of approximately 135,000 parameters, allowing it to process the 15 statistical features derived from the click data effectively.

The network operates by propagating input data through multiple hidden layers, with each layer transforming the data using activation functions. The output layer provides a binary classification (human vs. automated behavior) based on the learned representations. The feedforward architecture ensures that the information flows unidirectionally from input to output without forming cycles, optimizing it for tasks where the relationships between input features and output classes are complex but do not require temporal dependencies.

3.3 Data Collection

Data collection is a critical component of this project. We decided to use Minecraft, the click data used for training the model was gathered from two primary sources: human participants and artificial auto clicker tools. To ensure a balanced dataset, human-generated click data was obtained from staff members participating in a controlled environment, while auto clicker generated data was artificially created using popular automation tools.

The diversity in data collection provided a realistic representation of both legitimate human interaction and synthetic auto clicking behaviors. This mixed dataset allowed the model to generalize well

across various patterns of clicking, ensuring that it could accurately identify automated clicking in practical applications.

3.4 Data Preprocessing

To prepare the raw data for training the neural network, several preprocessing steps were applied:

- **Outlier Removal:** We removed unnecessary click intervals higher than 1000 milliseconds.
- **Feature Engineering:** The raw click intervals were transformed into 15 derived features (explained below) that capture various aspects of clicking behavior, such as central tendency, dispersion, and shape.
- **Data Splitting:** The dataset was split into training and validation sets, ensuring that the model could be evaluated on unseen data during the training process. This helped prevent overfitting and ensured that the model could generalize effectively to new click patterns.

3.5 Features Used

For the detection of auto clicking, we extracted a set of 15 statistical features from the click interval data. These features were chosen to capture a wide range of characteristics in the clicking behavior, providing the neural network with a rich dataset for training. The features are as follows:

- **Mean:** Represents the arithmetic average of the click intervals, reflecting the overall pace of clicking.
- **Median:** The middle value in the distribution of click intervals, providing a robust measure of central tendency.
- **Mode:** The most frequent click interval, indicative of repetitive behaviors, often seen in auto clickers.
- **Interquartile Range (IQR):** Measures the spread between the 25th and 75th percentiles of the click intervals, useful for detecting consistent vs. variable clicking patterns.

- **Entropy:** Quantifies the unpredictability of the click intervals, with lower entropy indicating regular, automated clicking.
- **Gini Coefficient:** Assesses inequality in the distribution of click intervals, with higher values suggesting irregularity.
- **Standard Deviation:** Provides a measure of variability around the mean, where human clicking typically results in higher variability than automated clicking.
- **Coefficient of Variation (CV):** Normalizes standard deviation by the mean, making it a more robust indicator of variability.
- **Max Value:** The largest click interval in the dataset, often a marker of outlier behavior.
- **Min Value:** The shortest time between clicks, helping to identify inhumanly fast responses characteristic of auto clickers.
- **Variance:** A measure of spread that captures the overall volatility in the click intervals.
- **Outliers Amount:** The number of significantly anomalous click intervals, providing insight into irregular clicking behavior.
- **Skewness:** A measure of asymmetry in the distribution of click intervals, where human data often skews differently than automated data.
- **Kurtosis:** Captures the “tailedness” of the distribution, indicating the presence of extreme click intervals.
- **Mid Range:** The average of the maximum and minimum values, offering a simplistic yet insightful summary of the dataset’s range.

These features provide a thorough statistical representation of click behaviors, allowing the neural network to learn both the common and uncommon characteristics of human and auto clicker patterns. This multi-dimensional feature set ensures that the model is capable of detecting even subtle deviations from normal human behavior.

4. RESULTS

4.1 Training Parameters

The feedforward neural network was trained using a standard backpropagation algorithm, with binary cross-entropy as the loss function. The model used Adam optimizer with β_1 of 0.9, β_2 of 0.999 and ϵ of 10^{-8} , adjusting weights iteratively based on the calculated error between the predicted and actual labels (human vs. auto clicker). The validation set, consisting of previously unseen data, was used to monitor the model's performance and ensure that it did not overfit to the training data.

4.2 Evaluation metrics

Accuracy: The ratio of correctly predicted instances (both positive and negative) to the total instances. It measures how often the model is right overall.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Instances}}$$

Precision: The ratio of correctly predicted positive instances (True Positives) to all instances predicted as positive (True Positives + False Positives). It shows how many of the predicted positives are actually correct.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

Recall: The ratio of correctly predicted positive instances (True Positives) to all actual positive instances (True Positives + False Negatives). It shows how well the model identifies actual positives.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

F1-Score: The harmonic mean of precision and recall. It balances the two, especially useful when classes are imbalanced (one class has many more instances than the other).

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Metric	Value
Accuracy	0.9424
Precision	0.9478
Recall	0.9364

F1 Score	0.9420
----------	--------

4.3 Confusion Matrix

The following table shows how the model classified the test data and how it was supposed to classify it, the values in this matrix are used to calculate the accuracy.

Legit	Cheat	What it should be
28405	1545	Legit
1906	28044	Cheat

In the first row, we can see that the model should classify all clicks as legit, and out of 30000 samples it classified 28405 correctly.

5. DISCUSSION

One promising area for future work is the inclusion of temporal features to capture time-dependent patterns in clicking behavior. Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks are better suited for analyzing sequential data, which could enhance detection accuracy by leveraging patterns that develop over time. This would allow the model to account for the evolution of clicking behavior instead of focusing solely on static statistics.

Expanding the dataset to cover a broader range of interaction types, such as mouse movements or key presses, would enhance the model's generalizability. A more diverse dataset would help the system detect a variety of automation behaviors across different applications, improving the robustness of the detection mechanism in new domains.

5.1 Strengths of the Neural Network Approach

The feedforward neural network's flexibility in handling high-dimensional input data gives it a strong advantage in learning patterns that may not be captured by traditional methods. The model generalizes well beyond the specific application of auto clicker detection and can be applied across

various domains where distinguishing between human and automated behaviors is necessary.

The scalability of the FFNN is another strength, as it is computationally efficient for making predictions once trained. This makes the architecture well-suited for high-throughput environments where real-time or near-real-time detection is critical.

5.2 Limitations and Challenges

One limitation is that the FFNN does not consider the temporal sequence of clicking behavior. Incorporating RNNs or LSTMs would allow the model to better capture time-related dependencies, potentially improving its accuracy for detecting subtle behavioral patterns.

The reliance on feature engineering is another constraint. While the statistical features we used provided strong initial results, feature extraction remains a manual and labor-intensive process. This could be mitigated by using Convolutional Neural Networks (CNNs) to automate feature extraction, particularly in scenarios where patterns in time-series data (such as click intervals) could be represented as a one-dimensional signal. CNNs could uncover patterns that may not be readily apparent through hand-engineered features.

Generalization to different environments remains a challenge as well. Although the current dataset and model perform well in controlled settings, further work is needed to ensure the model can handle real-world, noisy data from varied sources without overfitting to specific patterns.

6. CONCLUSION

This research demonstrates the effectiveness of neural networks in reliably detecting auto clickers, thanks to their ability to identify and model relationships between inputs. Our results show that neural networks offer a significant advantage over traditional systems in detecting human-like and bot-like patterns, providing better accuracy in discrimination.

7. REFERENCES

- [1] Mean
- [2] Median
- [3] Mode
- [4] Interquartile Range
- [5] Entropy
- [6] Gini Coefficient
- [7] Standard Deviation
- [8] Coefficient of Variation
- [9] Variance
- [10] Outliers
- [11] Skewness
- [12] Kurtosis
- [13] Mid range
- [14] 1D CNN